NCERT Learning Assistant

Project Domain: Education and Training
Project Type: Generative AI, Large Language Models, and Natural Language Processing
Submitted By: Mohammad Mamoor Ahmad
GitHub Link: https://github.com/mamoor2019/LLMBot

---

1. Introduction

The NCERT Learning Assistant is an AI-powered educational chatbot designed to help students understand school-level Science and Mathematics concepts using NCERT textbooks as the primary knowledge source.

Students frequently need additional explanations, examples, and step-by-step solutions while studying textbook chapters. Although general-purpose Large Language Models (LLMs) can answer many questions, their responses may not always align with the prescribed curriculum, use language appropriate to the student's grade, or remain grounded in the relevant textbook.

This project addresses these challenges by developing a domain-specific educational assistant using a pretrained LLM, textbook-based retrieval, and parameter-efficient fine-tuning.

The system is designed to retrieve relevant textbook content, generate student-friendly answers, and provide chapter and page references wherever the source metadata is available.

The initial scope focuses on NCERT Science and Mathematics for Classes 6–10. The architecture can subsequently be extended to Classes 11–12 and additional subjects.

2. Problem Statement

Students may struggle to understand scientific concepts, mathematical procedures, definitions, and textbook exercises. Traditional textbooks provide the prescribed material but do not interactively respond to individual doubts.

General-purpose AI chatbots offer interactive explanations, but they may introduce information outside the curriculum, generate incorrect mathematical solutions, or provide explanations that are too advanced for the student's class level.

The objective is to develop an interactive educational chatbot that combines the language-generation capabilities of a pretrained LLM with the factual grounding provided by NCERT textbook content.

3. Project Objectives

The primary objectives are:

1. Collect NCERT Science and Mathematics textbooks in PDF format.
2. Extract and preprocess textbook content while preserving class, subject, chapter, and page metadata.
3. Divide extracted content into meaningful text chunks suitable for retrieval and language-model processing.
4. Generate a question-and-answer dataset from the textbook passages.

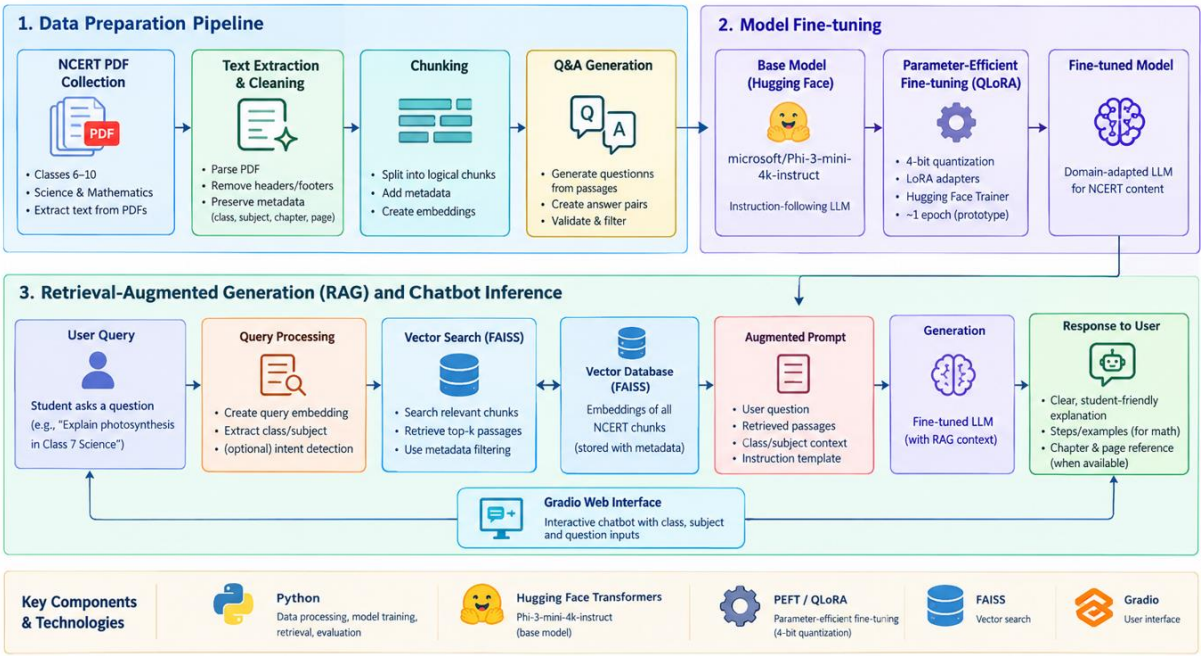
5. Select a pretrained instruction-following LLM that can be adapted within the available GPU memory.
6. Fine-tune the selected model using parameter-efficient techniques such as LoRA or QLoRA.
7. Implement a RAG pipeline to retrieve relevant NCERT content for student questions.
8. Develop an interactive chatbot interface.
9. Test the system using representative Science and Mathematics questions.
10. Compare the performance of different model configurations using a consistent evaluation dataset.

4. Technology Stack

Component	Technology	Purpose
Programming language	Python	Application and ML development
Development environment	Google Colab	Notebook-based experimentation
GPU	NVIDIA T4	Accelerated inference and fine-tuning
Data processing	Pandas, Python	Dataset preparation and manipulation
PDF extraction	PyMuPDF or pdfplumber	Extract textbook text
Language model	Hugging Face Transformers	Load and run the pretrained LLM
Tokenization	Hugging Face Tokenizers	Convert text into model tokens
Fine-tuning	PEFT and LoRA/QLoRA	Adapt the pretrained model efficiently
Quantization	bitsandbytes, where compatible	Reduce model memory requirements
Embeddings	Pretrained sentence-embedding model	Represent passages as vectors
Vector database	FAISS	Retrieve relevant textbook passages
Chatbot interface	Gradio	Provide an interactive user interface
Evaluation	Python, Pandas, human assessment	Measure response quality

5. Architecture Overview

The below architecture diagram shows the complete flow of the implementation and philosophy.



6. Development Environment Setup

6.1 GPU configuration

Google Colab provides a convenient environment for developing and testing the system. A Tesla T4 GPU with 16 GB of VRAM is a possible runtime configuration, subject to availability.

6.2 Dependency management

The development environment requires compatible versions of Transformers, Hugging Face Hub, PEFT, and other supporting libraries.

7. Data Collection

7.1 Data source

The primary data source is the official NCERT collection of school textbooks. The selected books cover Science and Mathematics for the intended classes.

Only the books included in the implemented project scope is considered as collected and processed.

7.2 Dataset organization

The textbook files are organized by class and subject to simplify processing and traceability.

The directory structure is:

LLMBot/

```
|
├── DataSet/
|   ├── ncert/
```

```
│   │   ├── Class_6/
│   │   │   ├── Science/
│   │   │   └── Mathematics/
│   │   ├── Class_7/
│   │   │   ├── Science/
│   │   │   └── Mathematics/
│   │   └── Class_8/
│   │   │   ├── Science/
│   │   │   └── Mathematics/
│   │   └── Class_9/
│   │   │   ├── Science/
│   │   │   └── Mathematics/
│   │   └── Class_10/
│   │   │   ├── Science/
│   │   │   └── Mathematics/
```

7.3 Data quality considerations

The collected data is checked for:

- Missing or unreadable PDF pages.
- Repeated headers and footers.
- Incorrect character extraction.
- Broken equations, tables, or scientific notation.
- Duplicate passages.
- Incorrect chapter boundaries.
- Missing source metadata.

Mathematics textbooks require particular attention because equations, fractions, superscripts, diagrams, and tables may not be represented accurately by ordinary PDF text extraction.

Where essential information is contained in images rather than extractable text, OCR or a separate visual-processing step may be required.

8. Text Extraction and Preprocessing

The PDF-processing stage converts textbook pages into machine-readable text. Each extracted record preserves information such as:

- Class

- Subject
- Book
- Chapter
- Chapter number
- Page number
- Document identifier
- Extracted text

The content is cleaned to remove irrelevant formatting while preserving the meaning of scientific explanations and mathematical expressions.

The cleaned text is then divided into manageable chunks.

8.1 Chunking strategy

Chunking divides long textbook chapters into smaller passages that can be processed efficiently by the LLM and indexed for retrieval.

A chunk contains enough context to support a meaningful question and answer without including excessive unrelated material.

9. Question-and-Answer Dataset Generation

9.1 Objective

The next stage converts textbook passages into educational question-and-answer examples.

Rather than manually writing every example, the pretrained LLM is used to generate one question and a concise answer from each selected NCERT passage.

This approach helps create a structured dataset for subsequent model fine-tuning.

9.2 Prompt design

The generation prompt includes the student's grade, subject, chapter, and the selected passage.

The model is instructed to:

- Generate one clear school-level question.
- Produce a concise answer supported by the passage.
- Return a JSON object with the keys question and answer.
- Avoid Markdown and unrelated information.

The instruction is designed to reduce unsupported content and maintain consistency across generated examples.

9.3 Generation function

The **generate_text()** function prepares the input using the tokenizer's chat template, transfers the inputs to the model's device, generates a response, and decodes the generated tokens.

The implementation uses **torch.inference_mode()** to avoid computing gradients during inference.

Generation settings include deterministic decoding with do_sample=False and a configurable maximum number of new tokens.

During performance optimization, cached generation was identified as an important consideration. Disabling the KV cache can substantially slow autoregressive generation, so cached generation should be enabled when compatible with the installed model and library versions.

9.4 Generating one Q&A record

The **generate_qa()** function constructs the prompt for a selected chunk, calls the generation function, extracts the JSON object, and validates the presence of both the question and answer.

Malformed JSON or missing fields cause the example to be rejected.

Valid records preserve the original class, subject, chapter, source, document identifier, page, and chunk identifier.

The resulting dataset contains fields such as:

Field	Description
instruction	Instruction governing the answer
input	Grade, subject, chapter, and passage
output	Generated answer
question	Generated student question
class	School class
subject	Science or Mathematics
chapter	Source chapter
source	Source document reference
document_id	Document identifier
page	Source page
chunk_id	Unique chunk identifier

9.5 Pilot experiment and checkpointing

The first question-generation experiment uses a sample limit of 20 chunks.

The code groups chunks by class and subject, samples representative passages, and processes the selected passages sequentially.

Checkpointing saves generated records periodically. Previously saved records are loaded when the notebook resumes, preventing successful examples from needing to be generated again.

This is particularly useful when model generation is slow or a Colab session is interrupted.

The initial pilot verifies that:

1. The model can process NCERT passages.
2. The output follows the requested JSON structure.
3. Questions are understandable and relevant.
4. Answers are supported by the passage.
5. The checkpointing mechanism works correctly.

The sample limit is a pilot setting, not the final dataset size. Dataset expansion should follow successful quality checks.

10. Pretrained Model Selection

10.1 Selection criteria

The model must be capable of following instructions, generating understandable explanations, and operating within the available GPU memory.

The selection criteria include:

1. Language-generation quality.
2. Ability to follow the requested response format.
3. Suitability for school-level Science and Mathematics.
4. Model size and inference memory requirements.
5. Compatibility with quantization and PEFT.
6. Availability of model weights and tokenizer.
7. License and usage conditions.
8. Compatibility with the chosen software environment.

10.2 Selected initial model

Microsoft Phi-3 Mini 4K Instruct was selected as the initial candidate because it is a relatively compact instruction-tuned model intended for general-purpose language tasks.

Its 4K context designation also requires attention when preparing long prompts: the combined prompt and generated output must fit within the model's context limit.

The model should be evaluated on actual NCERT questions before assuming that it is the best candidate.

Other models may be considered if they offer better educational performance, simpler setup, or improved compatibility with the available hardware.

10.3 Tokenizer loading

The tokenizer converts textual input into token IDs understood by the model. It also applies the model's expected chat formatting.

11. Fine-Tuning Using QLoRA

11.1 Purpose of fine-tuning

A pretrained LLM already understands language and can generate text. However, the desired chatbot must consistently behave like an educational assistant.

Fine-tuning aims to improve its ability to:

- Explain concepts using simple language.
- Produce concise, structured answers.
- Follow grade-level instructions.
- Provide step-by-step mathematical explanations.
- Generate examples and revision questions.
- Follow a defined answer format.
- Acknowledge when available evidence is insufficient.

Fine-tuning is not a substitute for factual retrieval. The model's learned parameters may not reliably reproduce every fact from the NCERT textbooks.

11.2 Why QLoRA?

Full fine-tuning updates the model's original parameters and can require substantial GPU memory.

QLoRA reduces memory requirements by loading the base model in low-precision quantized form and training small adapter parameters.

The principal components are:

- **4-bit quantization:** Reduces the memory footprint of the base model.
- **LoRA adapters:** Introduce trainable low-rank parameter updates.
- **PEFT:** Provides the framework for training and managing these adapters.
- **Transformers:** Loads the model and implements training and inference.

The approach is suitable for experimentation on a constrained GPU environment, although the exact configuration depends on model compatibility and sequence length.

11.3 Fine-tuning data format

The generated question-and-answer records is converted into the training format expected by the selected model.

An illustrative example is:

{

"instruction": "Answer the question using NCERT content.",

"input": "Explain why plants need sunlight.",

"output": "Plants use sunlight to carry out photosynthesis..."

}

The final training format should match the model's chat template and the selected fine-tuning library.

The training examples teach the desired answer behavior, while retrieval supplies the relevant textbook evidence during chatbot inference.

11.4 Training and validation

The training dataset is used to update the adapter parameters. The validation dataset is used to monitor performance and identify overfitting.

11.5 Saving the model

The final artifacts include the trained adapter weights, adapter configuration, base model identifier, tokenizer configuration where needed, and training settings.

12. Retrieval-Augmented Generation (RAG)

12.1 Why RAG is required

Fine-tuning teaches the model how to respond, but it does not guarantee that the model will reliably recall every textbook fact.

RAG addresses this limitation by retrieving relevant NCERT passages at question time and providing them to the LLM as context.

This enables answers to be grounded in source material and makes it possible to display textbook references.

12.2 Building the knowledge base

The cleaned NCERT chunks are transformed into vector embeddings using FAISS. Each indexed passage retains its source metadata.

The retrieval pipeline is:

1. Receive the student's question.
2. Encode the question as an embedding.
3. Search for semantically similar textbook chunks.
4. Apply class, subject, and optionally chapter filters.
5. Select the most relevant passages.
6. Provide the retrieved text to the language model.
7. Generate an answer grounded in the supplied context.
8. Return the answer with source information.

12.3 Answer-generation prompt

The prompt should instruct the model to use the retrieved NCERT passages and acknowledge when those passages do not provide sufficient evidence.

An example instruction is:

You are an NCERT educational assistant. Answer the student's question using the supplied NCERT material. Explain the concept in language appropriate to the selected class. For mathematical questions, show the solution steps clearly. If the supplied material does not support the answer, state that the available NCERT content is insufficient rather than inventing information.

13. Chatbot Application Development

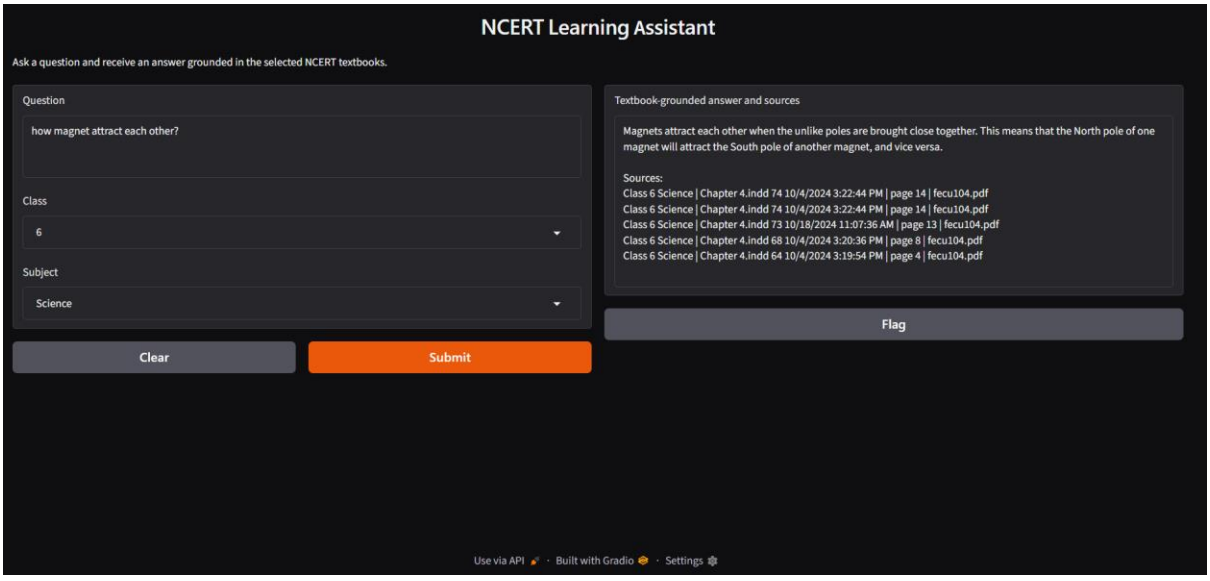
13.1 User interface

Gradio is used for the interactive interface because it supports rapid development of model demonstrations in Python.

The interface provides:

- Class selection.
- Subject selection.
- A text box for the student's question.
- An option to request a simple explanation, example, or quiz.
- A response area.
- Relevant NCERT chapter and page references.

  Following is the UI with one sample question and answer.



13.2 Request-processing flow

When a student submits a question, the application follows the below steps:

1. Validate the selected class and subject.
2. Retrieve relevant passages from the NCERT vector database.

3. Construct a prompt using the question and retrieved passages.
4. Run the fine-tuned model.
5. Decode and format the answer.
6. Display the response and source references.

14. Testing Strategy

Testing should cover the complete system, not just whether the model produces fluent text.

14.1 Functional testing

Test	Expected behaviour
Load model and tokenizer	Both load without errors
Submit a valid question	The chatbot generates a response
Select another class	Retrieval respects the selected class
Select another subject	Retrieval uses the appropriate subject
Ask a chapter-specific question	Relevant chapter content is retrieved
Ask a Science question	The answer explains the concept clearly
Ask a Mathematics question	The response includes logical solution steps
Ask an unsupported question	The system acknowledges insufficient evidence when appropriate
Display source references	The displayed references match retrieved metadata
Restart the application	Saved model and index can be loaded again

14.2 Dataset testing

The generated Q&A dataset is tested for schema consistency, missing values, duplicates, malformed records, and source traceability.

The checkpoint file should also be tested to confirm that interrupted processing can resume without unintentionally duplicating examples.

14.3 Retrieval testing

Retrieval quality can be evaluated using a set of questions for which the relevant source chapter or passage is known.

Useful measures include:

- Recall@k: whether a relevant passage appears among the top-k retrieved results.
- Precision@k: how many of the retrieved passages are relevant.
- Mean Reciprocal Rank (MRR): how highly the first relevant passage is ranked.

These measures help distinguish retrieval failures from generation failures.

15. Experimental Design

To understand the contribution of each technique, compare the following configurations using the same test questions.

Experiment A: Base LLM

The pretrained model answers questions without textbook retrieval or project-specific fine-tuning.

Experiment B: Base LLM + RAG

The base model receives relevant retrieved NCERT passages.

Experiment C: Fine-tuned LLM

The fine-tuned model answers questions without retrieval.

Experiment D: Fine-tuned LLM + RAG

The fine-tuned model receives relevant NCERT passages and generates answers using the learned educational response style.

The comparison should use the same held-out questions and consistent evaluation procedures. Record inference settings and any differences in prompts.

The principal metrics should include answer correctness, relevance, completeness, grade appropriateness, NCERT alignment, retrieval quality, and the frequency of unsupported claims.

Where possible, use independent human reviewers and a predefined scoring rubric. Automated similarity metrics may supplement this assessment but should not be treated as definitive measures of factual correctness.

16. Challenges

16.1 Library compatibility

16.2 Slow text generation

16.4 Limited GPU memory

.

16.5 Mathematical formatting

17. Limitations

The chatbot's reliability depends on the quality and completeness of the extracted textbook content, the relevance of retrieved passages, and the language model's reasoning capabilities.

Text-only extraction may not preserve all diagrams, tables, or equations. Fine-tuning on generated examples may introduce errors if the examples are not validated. Retrieval also cannot guarantee correctness if the relevant passage is not indexed or is ranked poorly.

The initial implementation is limited to the books and subjects actually processed. Results from a limited pilot should not be generalized to every NCERT class or subject.

The chatbot is an educational support tool and should not be presented as a replacement for teachers, textbooks, or independent verification of important answers.

19. Future Enhancements

Potential improvements include:

1. Extending the knowledge base to Classes 11–12.
2. Adding Biology, Physics, Chemistry, and additional school subjects where applicable.
3. Supporting diagrams and image-based textbook questions.
4. Adding automated mathematical answer verification.
5. Improving retrieval through hybrid keyword and semantic search.
6. Adding multilingual explanations.
7. Supporting personalized quizzes and revision plans.
8. Evaluating additional small instruction-tuned LLMs.
9. Measuring performance across classes and subjects separately.
10. Improving deployment, latency, and model monitoring.

20. Conclusion

The NCERT Learning Assistant demonstrates a practical approach to applying generative AI to school-level education. The proposed system combines a pretrained LLM, parameter-efficient fine-tuning, and retrieval from NCERT textbooks to support interactive Science and Mathematics learning.

The data pipeline transforms textbook PDFs into structured passages and educational Q&A records. Fine-tuning aims to improve the model's response style and educational behavior, while RAG provides relevant source content at inference time. A Gradio interface makes these capabilities accessible through an interactive chatbot.

The project's central research contribution is the comparative evaluation of a general-purpose LLM, retrieval-augmented answering, domain-specific fine-tuning, and their combination.

The project therefore provides both an implementable educational application and a framework for studying how fine-tuning and retrieval can improve curriculum-focused question answering.